Транзакционная память: история и развитие

14 мин

49K

Высоконагруженные системы*C++*Параллельное программирование*



Определение

Параллельное программирование сложно. При использовании систем с общей памятью не обойтись без синхронизации доступа параллельных процессов/потоков к общему ресурсу (памяти). Для этого используются:

- блокировки (mutex);
- алгоритмы без блокировки (lockless, lock-free);
- транзакционная память.

Транзакционная память — технология синхронизации конкурентных потоков. Она упрощает параллельное программирование, выделяя группы инструкций в атомарные

транзакции. Конкурентные потоки работают параллельно[1], пока не начинают модифицировать один и тот же участок памяти. К примеру, операции добавления узлов в красно-чёрное дерево (анимация в заголовке) способны работать параллельно в нескольких потоках.

Скрытый текст

Подход к управлению конкурентностью с использованием транзакционной памяти называют оптимистичным: мы считаем, что потоки работают независимо друг от друга и в редких случаях изменяют одни и те же данные. В таком случае большинство транзакций заканчивается успешно. В противоположность этому, подходы с использованием блокировок пессимистичны: мы полагаем, что потоки будут конфликтовать всегда и всегда запрещаем им находиться в критической секции одновременно.

Если происходит конфликт данных, транзакция отменяется. Отмена транзакции приводит к отмене действий, которые совершал поток за время транзакции. После этого транзакция обычно перезапускается, либо вызывается функция, заранее указанная как «запасной выход», чаще всего откат на использование блокировок.

Плюсы транзакционной памяти:

- относительная простота использования (заключение целых методов в блок транзакции);
- полное отсутствие блокировок и взаимных блокировок;
- повышение уровня параллелизма, а следовательно, и производительности.

Транзакционная память — не серебряная пуля. Есть и минусы:

- при неправильном использовании возможно падение производительности и некорректная работа;
- ограниченность применения — в транзакции нельзя выполнять операции, действие от которых невозможно отменить;
- сложность отладки — поставить breakpoint внутри транзакции невозможно.

Рождение технологии

Транзакции в базах данных существуют уже несколько десятилетий. Впервые идея переноса транзакций из мира баз данных в мир параллельного программирования возникла в 1980-х. Развивали и популяризовали технологию Maurice Herlihy, Ravi Rajwar, Nir Shavit. Первые исследования предлагали аппаратные реализации транзакционной памяти, которым не суждено было родиться ещё 30 лет.
В 1990-х появились первые программные реализации технологии, аппаратные реализации подтянулись к 2000-ым.

Программные реализации (Software Transactional Memory, STM)

Среди множества реализаций программной транзакционной памяти я хотел бы выделить четыре. Примеры доступны на github: JIghtuse/tm-experiments.

Clojure

Clojure — единственный язык, ядро которого поддерживает транзакционную память. Основные конструкции STM: ref (ссылка на данные, через которую данные меняются только в транзакции) и dosync (блок транзакции).

Подход к STM в Clojure называется управлением конкурентным доступом с помощью многоверсионности (MultiVersion Concurrency Control, MVCC): хранятся множественные логические версии данных, используемых в транзакции. В течение транзакции поток наблюдает снимок данных на момент её начала.

Скрытый текст

Рассмотрим пример перевода средств между банковскими аккаунтами:

Скрытый текст

Существуют варианты использования конкурентности, при которых среде позволяется «расслабиться» для достижения дополнительной производительности. К примеру, представьте что вы храните журнал транзакций за день. Порядок транзакций в журнале не важен, если вы знаете, что конечный баланс будет корректным. Если вы получаете два взноса в $100 и $50, очерёдность внесения их в журнал не играет роли. Взнос от двух транзакций коммутативен, и clojure предоставляет конкурентную операцию commute как раз для таких случаев.

Скрытый текст

Haskell

Транзакционная память в Haskell содержится в библиотеке STM, которая входит в Haskell Platform. Некорректное использование транзакционных типов определяется на этапе компиляции программы.

Haskell обладает мощной системой типов. Язык разделяет функции с побочными эффектами и чистые функции. Любое значение типа IO t называется действием. Для выполнения какого-либо действия атомарно в Haskell, действие предваряется словом atomically. Вызов atomically с действием в качестве аргумента даёт две гарантии:

- атомарность — эффект от atomically act будет виден другим потокам сразу и целиком. Ни один другой поток не способен увидеть состояние внутри атомарного действия, лишь конечное состояние.
- изоляция — во время вызова atomically act действие act выполняется абсолютно независимо от других потоков. Оно снимает состояние мира при запуске и выполняется в этом состоянии.

atomically имеет тип atomically :: STM a -> IO a. Действие типа STM a — аргумент. Как и действие IO, действие STM имеет побочные эффекты, но их диапазон значительно уже. В основном в действии STM происходит запись или чтение транзакционной переменной TVar a:

- readTVar :: TVar a -> STM a
- writeTVar :: TVar a -> a -> STM ()

Рассмотрим пример перевода средств между аккаунтами банка.

Скрытый текст

Scala

Реализация STM для Scala (ScalaSTM) разрабатывалась под впечатлением от реализаций в Haskell и Clojure. Помимо Scala, ScalaSTM вызывается из Java и Clojure. Реализация используется в популярном фреймворке для написания параллельных программ Akka.

ScalaSTM предоставляет ячейку Ref, изменяемую исключительно внутри транзакции. Структуры данных на основе неизменяемых объектов и Ref используются многими потоками.

Рассмотрим реализацию двусвязного потокобезопасного списка с использованием транзакционной памяти. К сожалению, собрать пример на Scala мне не удалось, оставляю это занятие читателю.

Скрытый текст

Scala объединяет идеи многих парадигм программирования. Неудивительно, что и в области транзакционной памяти язык имеет самобытные технологии. Вышеупомянутый фреймворк Akka объединяет возможности акторов и транзакционной памяти, получая агенты и транзакторы, которые окончательно взорвут вам мозг.

C/C++ (GCC 4.7+)

Начиная с версии 4.7, GCC поддерживает транзакционную память. Реализация представляет собой библиотеку времени выполнения libitm, для компиляции указывается флаг -fgnu-tm (-mrtm, -mhle). Библиотека разрабатывалась с оглядкой на черновик транзакционных конструкций для C++ (предлагается включение в стандарт языка).

Большинство реализаций аппаратной транзакционной памяти используют принцип наибольшего усилия. Поэтому практичные реализации используют объединение технологий аппаратурной и программной транзакционной памяти. Такие системы называют системами «гибридной транзакционной памяти». К ним относится, в частности, реализация GCC.

В язык вводятся ключевые слова:

- __transaction_atomic { … } — указание, что блок кода — транзакция;
- __transaction_relaxed { … } — указание, что небезопасный код внутри блока не приводит к побочным эффектам;
- __transaction_cancel — явная отмена транзакции;
- attribute((transaction_safe)) — указание транзакционно-безопасной функции;
- attribute((transaction_pure)) — указание функции без побочных эффектов.

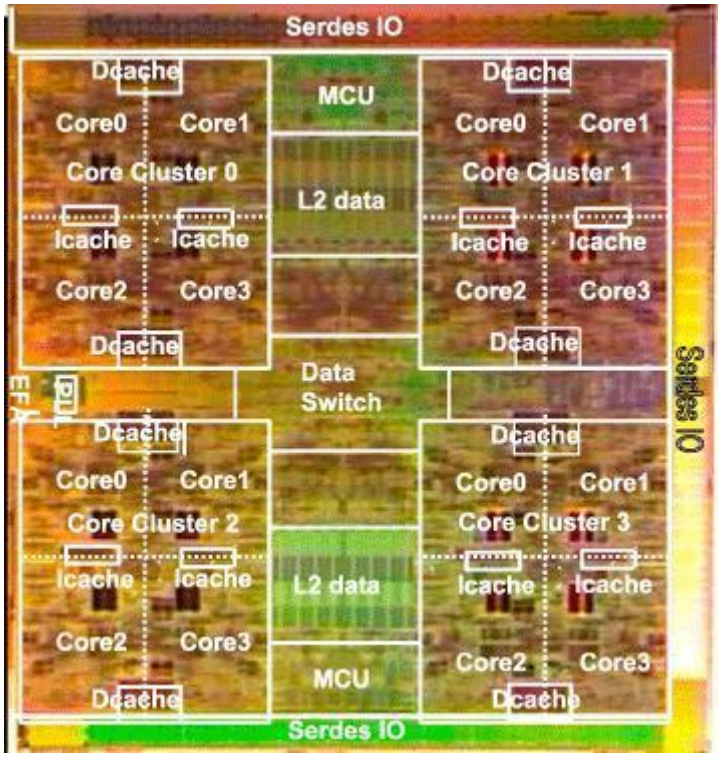
Для демонстрации использования транзакционной памяти в C будем заполнять гистограмму данных в конкурентных потоках.

Скрытый текст

Аппаратные реализации (Hardware Transactional Memory, HTM)

Лишь в последнее время начали появляться аппаратные реализации транзакционной памяти.

Sun Rock (SPARC v9) 2007-2008



Микропроцессор Rock от компании Sun Microsystems был первым микропроцессором с аппаратной поддержкой транзакционной памяти. 64-разрядный процессор архитектуры SPARC v9 имел 16 ядер.

В 2007 компания объявила о поддержке технологии. Для функционирования транзакционной памяти были добавлены две новые инструкции chkpt и commit и один новый статусный регистр cps. Инструкция chkpt <fail_pc> использовалась для начала

транзакции, а инструкция commit для её завершения. При отмене транзакции происходил переход на <fail_pc>, в это время можно было проверить регистр cps для определения причины отмены. Транзакции прерывались по причинам конфликтов данных, промахов TLB, прерываний, сложных инструкций. Тем не менее, во многих случаях использование транзакционной памяти в процессоре Rock давало преимущества в синхронизации.

В 2008 инженеры Sun представили интерфейс транзакционной памяти и симулятор Adaptive Transactional Memory Test Platform. После покупки компании Sun корпорацией Oracle проект Rock был закрыт: «Этот процессор имел два потрясающих свойства: он был невероятно медленным и потреблял громадное количество энергии. Он был настолько горячим, что им пришлось поставить сверху 12 дюймов охлаждающих вентиляторов, чтобы процессор не перегревался. Было бы безумием продолжать этот проект.»[2]

IBM BlueGene/Q (PowerPC A2) 2011



Суперкомпьютер Sequoia с архитектурой BlueGene/Q стал первой коммерческой системой с аппаратной поддержкой транзакционной памяти. Технология работает в 32-мегабайтном кэше второго уровня процессора PowerPC A2 (PowerPC BQC 16C). Данные в кэше имеют метку “версия”, и кэш способен хранить несколько версий одних и тех же данных.

Программа сообщает процессору о начале транзакции, делает свою работу и сообщает,

что транзакцию нужно завершить (commit). В случае если другие потоки изменяли те же данные — создавая версии — кэш отклоняет транзакцию и поток пытается провести её вновь. Если других версий данных не появилось, данные модифицируются и транзакция завершается успешно.

Технология версионных меток в PowerPC A2 также используется для спекулятивного выполнения. Вместо ожидания новой версии данных поток может выполнить вычисления с имеющимися данными, спекулятивно производя полезную работу. Если данные были такими же, как и после обновления, поток завершает (commit) работу из кэша. Производительность выше: поток выполнил работу до получения финального значения. В противном случае результаты спекулятивной работы отклоняются и поток производит вычисления с корректными значениями.

Поддержка транзакционной памяти — в некотором роде логическое расширение возможности, давно присутствующей в процессорах PowerPC — “load-link/store-conditional”, или LL/SC. LL/SC — примитивная операция, которая может использоваться как строительный блок для всех потокобезопасных конструкций. Первая часть LL/SC — load-link — используется программой для получения данных из памяти. Далее поток изменяет данные и записывает их обратно в память с помощью store-conditional. Команда завершается успешно, если данные не менялись. В противном случае программа повторяет действия с данными с начала.

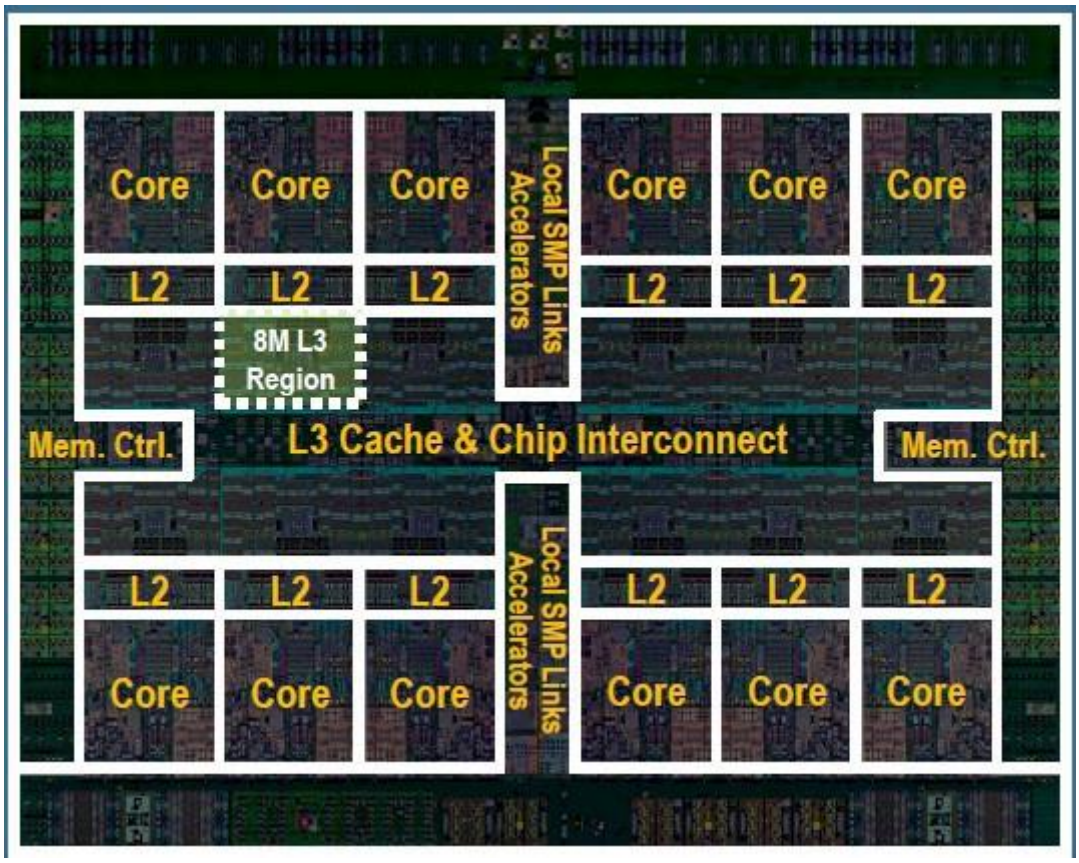
Транзакционная память — LL/SC на стероидах: потоки выполняют операции LL на множестве различных областей памяти, а операция завершения транзакции атомарно изменяет все области памяти или отменяет транзакцию (как SC).

Ruud Haring, который представил работу IBM по транзакционной памяти на Hot Chips, признался, что при реализации компания столкнулась со множеством нетривиальных проблем. При всей сложности, реализация имеет ограничения: она не предоставляет межпроцессорной поддержки транзакций. Проблема не актуальна для Sequoia и в то же время позволяет оценить выигрыш от использования транзакционной памяти.

IBM zEnterprise EC12 (Z/Architecture) 2012

Первый коммерческий сервер с поддержкой инструкций транзакционной памяти. При использовании транзакционной памяти IBM обнаружила рост производительности в 45% по сравнению с машиной z196 в базе данных DB2 и работах в виртуализированных серверных нагрузках.

IBM Power8 (Power) 2013



Контроллеры памяти Power 8 поддерживают транзакционную память. Поддержка технологии в ядре Linux появилась начиная с версии ядра 3.9.
Информации по HTM в Power8 удалось найти не так много, надеюсь она ещё появится.

Intel Haswell (x86) 2013



В 2012 компания Intel объявила о введении расширений транзакционной синхронизации (Transactional Syncrhonization Extensions, TSX) в набор инструкций x86. Расширения позволяют программистам помечать отдельные участки кода как транзакции.
В 2013 с выходом поколения процессоров Haswell аппаратная поддержка транзакционной памяти впервые становится доступной на потребительском уровне.

Haswell управляет наборами чтения и записи с гранулярностью линии кэша, отслеживая адреса кэша данных L1. Конфликты определяются с помощью протокола когерентности кэша. Что логично предположить, поскольку задачи определения конфликтов транзакций и поддержки когерентности кэшей очень близки: если значение меняется в одном потоке, но не в других, то что-то неладно.

TSX состоит из двух частей. Аппаратное исключение блокировок (Hardware Lock Elision, HLE) предоставляет простую конвертацию программ на основе блокировок в транзакционные программы, причём полученные программы будут обратно совместимы с существующими процессорами. Ограниченная транзакционная память (Restricted Transactional Memory, RTM) является более полной реализацией транзакционной памяти.

Hardware Lock Elision

HLE слегка модифицирует инструкции для изменения участка памяти. Технология добавляет префиксы — инструкции, не производящие каких-либо действий, но меняющие

интерпретацию следующей инструкции — для модификации инструкций взятия и освобождения блокировки (XACQUIRE и XRELEASE соответственно).

Между взятием и освобождением блокировки процессор отслеживает участки памяти, которые читают и записывают потоки. В случае конфликта — если два потока модифицируют одни данные, или один поток читает данные, которые другой записывает — процессор отменяет транзакцию при освобождении блокировки. В противном случае, выполнение продолжается нормально.

Таким образом, HLE позволяет общепринятому коду на основе блокировок работать оптимистично. Каждый поток будет считать, что получил блокировку, в то время как остальные потоки могут работать одновременно. До тех пор пока это безопасно, транзакции не отменяются.

Технология обратно совместима с процессорами без поддержки HTM. Операции по управлению блокировкой остаются, но со специальным префиксом. Процессоры Haswell будут учитывать префикс и использовать транзакционное исполнение вместо манипуляций с блокировками. Любой другой процессор будет игнорировать префикс и просто управлять блокировкой, используя традиционное поведение на основе блокировок. Инструкции XACQUIRE и XRELEASE уже существуют, но не несут какого-либо смысла, пока не используются со специфичными инструкциями.

HLE позволяет писать программы и операционные системы, которые будут использовать транзакции на Haswell, повышая уровень параллелизма без блокировок. Код будет работать корректно на текущих процессорах. В итоге, ввод новой возможности будет простым и безопасным.

HLE на простом примере

Restricted Transactional Memory

RTM требует большего участия: он уходит от обратной совместимости и вводит три новых инструкции. В то время как HLE неявно использует транзакции, позволяя коду на основе блокировок работать параллельно, RTM делает начало, завершение и прерывание транзакций явными. Поток начинает транзакцию инструкцией XBEGIN, предоставляя “запасную” функцию, которая запускается в случае прерывания транзакции. Транзакция завершается инструкцией XEND, при этом процессор проводит транзакцию если не было конфликтов или прерывает её, переходя в запасную функцию. Транзакции явно прерываются в программе инструкцией XABORT.

Благодаря явному использованию границ и “запасному выходу”, RTM позволяет контролировать транзакции полнее, чем HLE. В долгосрочной перспективе RTM упростит реализацию возможностей транзакций.

Выводы

Использование транзакционной памяти — жизнеспособная альтернатива существующим методам синхронизации, которая позволяет упростить параллельное программирование. *О неточностях перевода, стилистики, и об опечатках просьба писать в личном сообщении.*

Примечания

1. О разнице понятий «конкурентность» и «параллелизм» грамотно выразился Rob Pike в выступлении "Concurrency is not Parallelism": «Конкурентность — работа с множеством вещей за один раз, параллелизм — выполнение множества вещей за один раз»
2. Да-да, помимо пачки отличных программных продуктов (OpenSolaris, MySQL, OpenOffice) Oracle забросила и перспективную аппаратную технологию

Да, в языке C# и платформе .NET существует концепция транзакционной памяти (Software Transactional Memory, или **STM**). Она позволяет управлять параллелизмом, объединяя несколько операций с данными в одну **атомарную транзакцию** — если одна из операций завершается ошибкой или сталкивается с конфликтом, вся транзакция откатывается (как в базах данных).

В официальном рантайме .NET **нет встроенной из коробки поддержки STM**, так как Microsoft в свое время закрыла экспериментальный проект *MSR STM.NET*. Однако эту концепцию можно полноценно использовать с помощью сторонних библиотек (например, **Nito.Volatile** или **Stm.Net**) или написать базовый механизм самостоятельно для учебных целей.

Ниже приведено подробное объяснение механизмов и пример реализации.

Как работает механизм STM?

Вместо использования блокировок (классический lock), которые могут приводить к взаимным блокировкам (Deadlocks), STM использует **оптимистичный подход**:

1. **Чтение и запись в локальный журнал:** Поток выполняет все операции внутри транзакции в своем изолированном пространстве (лог транзакции). Внешний мир этих изменений пока не видит.
2. **Проверка конфликтов (Validation):** В момент завершения транзакции система проверяет, не изменил ли другой поток те же самые данные, пока выполнялась текущая трансляция.
3. **Фиксация (Commit) или Откат (Rollback):**
    - Если конфликтов нет, данные атомарно записываются в общую память.
    - Если конфликт обнаружен, транзакция **откатывается**, а лог очищается. Обычно после этого поток автоматически пытается выполнить транзакцию заново (Retry).

Пример реализации STM на C# (Концептуальный)

Поскольку готового ключевого слова transaction в C# нет, механизм реализуется через паттерн «Единица работы» (Unit of Work) или специальные обертки над переменными.

Вот пример того, как выглядит транзакционный перевод денег между счетами с использованием гипотетической или кастомной STM-библиотеки:

csharp

```
using System;
using System.Threading.Tasks;

namespace TransactionalMemoryExample
{
  // Обертка над значением, поддерживающая транзакционность
  public class TransactionalRef<T>
  {
    public T Value { get; set; } // В реальной STM здесь будет логика версионирования

    public TransactionalRef(T initialValue)
    {
```

```
        Value = initialValue;
    }
}

public class BankAccount
{
    // Баланс хранится в транзакционной переменной
    public TransactionalRef<decimal> Balance { get; }

    public BankAccount(decimal initialBalance)
    {
        Balance = new TransactionalRef<decimal>(initialBalance);
    }
}

public static class TransactionManager
{
    // Симуляция выполнения транзакции с автоматическим перезапуском при конфликте
    public static void ExecuteTransaction(Action action)
    {
        bool success = false;
        while (!success)
        {
            try
            {
                // 1. Открываем транзакцию (в реальности: создание лога потока)
                // StartTransaction();

                action();
```

```
            // 2. Пытаемся зафиксировать изменения (Commit)
            // If conflicts detected -> throw ConflictException
            success = true;
            Console.WriteLine($"[{Task.CurrentId}] Транзакция успешно зафиксирована.");
        }
        catch (Exception) // Например, ConflictException
        {
            Console.WriteLine($"[{Task.CurrentId}] Конфликт! Откат и повторная попытка...");
            // 3. Откат изменений (Rollback) автоматически происходит, так как данные не записаны в основной Ref
        }
      }
    }
  }

  class Program
  {
    static void Main(string[] args)
    {
      var accountA = new BankAccount(1000 activate);
      var accountB = new BankAccount(500);

      // Параллельно запускаем перевод денег
      Parallel.Invoke(
        () => TransferMoney(accountA, accountB, 200),
        () => TransferMoney(accountB, accountA, 50)
      );
```

```
        Console.WriteLine($"Итоговый баланс A: {accountA.Balance.Value}");
        Console.WriteLine($"Итоговый баланс B: {accountB.Balance.Value}");
    }

    public static void TransferMoney(BankAccount from, BankAccount to, decimal amount)
    {
        // Код внутри этого блока выполняется атомарно, изолированно и согласованно
        TransactionManager.ExecuteTransaction(() =>
        {
            if (from.Balance.Value < amount)
            {
                throw new InvalidOperationException("Недостаточно средств");
            }

            // Изменения записываются в транзакционный контекст
            from.Balance.Value -= amount;
            to.Balance.Value += amount;
        });
    }
  }
}
```

Используйте код с осторожностью.

Сравнение: STM против классического Lock

Критерий	**Классический lock (Пессимистичный)**	**Транзакционная память (Оптимистичный)**
Deadlocks (Взаимные блокировки)	Возможны, если перепутать порядок взятия локов.	**Невозможны в принципе.**

Поведение при отсутствии конфликтов	Тратит ресурсы на захват/освобождение монитора.	Работает максимально быстро без задержек.
Поведение при высокой конкуренции	Потоки выстраиваются в очередь (блокируются).	Потоки часто перезапускаются (высокая нагрузка на CPU / Спины).
Сложность масштабирования	Очень сложно поддерживать в больших системах.	Легко масштабируется, код выглядит «чистым».

Почему STM не стала стандартом в .NET?

1. **Откат побочных эффектов:** Если внутри транзакции вы отправили Email или записали лог в файл, STM не сможет это «откатать» назад при конфликте.
2. **Накладные расходы:** Ведение логов чтений и записей для каждого потока создает серьезную нагрузку на память и сборщик мусора (Garbage Collector).
3. **Альтернативы:** В .NET прижились другие подходы к параллелизму — неблокирующие структуры данных (ConcurrentDictionary), модель акторов (библиотека *Akka.NET* или *Proto.Actor*) и каналы (System.Threading.Channels).

Если вам интересно глубоко изучить эту тему в продакшн-ключе для .NET, рекомендую посмотреть в сторону архитектурного паттерна **Event Sourcing** и библиотек для **Sagas (Car)** в распределенных транзакциях (например, *MassTransit*), которые решают похожую задачу, но на уровне бизнес-логики.

Если вы хотите спроектировать конкретный алгоритм, уточните: вам необходима **локальная работа с памятью** (внутри одного приложения) или организация транзакций между **микросервисами**?